

```

    minitinerary = itinerarytoreturn
    minidx = time

    if(abs(time - maxitin) <= 1):
        itinerarytoreturn = minitinerary.copy()

    return(itinerarytoreturn.copy())

```

[Listing 6-6](#): This function performs the full simulated annealing process and returns an optimized itinerary.

Here, we define global variables for the minimum distance achieved so far, the itinerary that achieved it, and the time at which it was achieved. If the time progresses very far without finding anything better than the itinerary that achieved our minimum distance, we can conclude that the changes we made after that point were mistakes, and we allow resetting to that best itinerary. We'll reset only if we've attempted many perturbations without finding an improvement on our previous best, and a variable called `resetthresh` will determine how long we should wait before resetting. Finally, we add a new argument called `maxitin`, which tells the function how many total times we intend to call this function, so that we know where exactly in the process we are. We use `maxitin` in our temperature function as well so that the temperature curve can adjust flexibly to however many perturbations we intend to perform. When our time is up, we return the itinerary that gave us the best results so far.

Testing Our Performance

Now that we have made these edits and improvements, we can create a function called `siman()` (short for simulated annealing), which will create our global variables, and then call our newest `perturb()` function repeatedly, eventually arriving at an itinerary with a very low traveling distance ([Listing 6-7](#)).

```

def siman(itinerary,cities):
    newitinerary = itinerary.copy()
    global mindistance
    global minitinerary
    global minidx
    mindistance = howfar(genlines(cities,itinerary))

```